

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer Architecture

Dado el requerimiento de máximo 2 archivos, se adopta una arquitectura ultra-condensada donde cada capa (frontend/backend) reside en un único artefacto autocontenido. Este patrón es válido para herramientas internas de baja escala y alta velocidad de entrega.

CAPAS DEL SISTEMA:

1. Presentación (index.html): SPA estática sin framework JS. Tailwind CSS vía CDN elimina la necesidad de build tools. La lógica de estado (loading, error, results) se gestiona directamente en el DOM mediante JavaScript vanilla con fetch API asíncrona.
2. Procesamiento (main.py): Microservicio FastAPI con un único endpoint POST /api/analyze. La simulación de latencia usa `asyncio.sleep()` para no bloquear el event loop de uvicorn. El cálculo de tokens es determinístico: $\text{len}(\text{words}) * 1.3$.

SEGURIDAD Y CORS:

Al ser una herramienta interna consumida desde `file://` o `localhost`, se configura `CORSMiddleware` con `origins=["*"]` o restringido a `localhost` puertos comunes (3000, 5500, 8080). No se exponen datos sensibles ni se conecta a servicios externos. Sin autenticación requerida dado el contexto interno.

FLUJO DE DATOS:

Usuario escribe prompt !' Validación client-side (campo vacío !' borde rojo, no fetch) !' POST JSON {prompt: string} !' FastAPI valida payload con Pydantic BaseModel !' `asyncio.sleep(random 0.5-2s)` !' Cálculo tokens = $\text{round}(\text{len}(\text{prompt.split()})) * 1.3$!' Response JSON {tokens, latency_ms, status} !' Renderizado de 3 tarjetas en el DOM.

LÓGICA DE NEGOCIO - Estado del prompt:

- tokens <= 1000: Estado = 'Óptimo' (verde)
- tokens > 1000: Estado = 'Alerta' (amarillo/rojo)

DEPENDENCIAS MÍNIMAS:

- Backend: fastapi, uvicorn[standard], pydantic (incluido en fastapi)
- Frontend: Tailwind CSS CDN (sin instalación)

ESTRATEGIA DE EJECUCIÓN:

1. Ejecutar: `uvicorn main:app --reload --port 8000`
 2. Abrir index.html directamente en el navegador (`file://`) o con Live Server
- No se requiere Docker, CI/CD ni base de datos para esta herramienta interna.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 14,074

Tokens de salida (output): 7,169

Total tokens: 21,243

Horas-hombre estimadas: ~2.5 horas

Modelo utilizado: claude-sonnet-4-6